GRADUATION PROJECT · INFORMATION TECHNOLOGY · 2025/2026

CANvisionNative

A diagnostic and validation platform for the CAN traffic of Battery Management Systems



PRESENTED BY

Mohamed Mootaz Ben Kraiem

ACADEMIC SUPERVISOR

Mrs Nadia Boulifa

ESPRIT

CORPORATE SUPERVISOR

Mr Hamda Guizani

Capgemini Engineering

Summary

01 Context
The host company and how a CAN bus works

02 Problem
What is missing today and the existing tools

03 Objectives and requirements
The objectives, the method and the requirements

04 Proposed solution
Architecture, detection, models and explanation

05 Demonstration
The real application on real recordings

06 Validation
Two iterations on real recordings

07 Limits
What is not proven yet

08 Conclusion
Results and next steps

0x100 · CONTEXT

Who is the host company?

01 02 03 04 05 06 07 08

Who is the host company?

Capgemini engineering

The engineering and research and development branch of the Capgemini group, which offers consulting and engineering services to many industries, the automotive sector among them.

CANvisionNative is not a team deliverable. I started it in February 2026, alongside my job, from the difficulties of my daily validation work.

Automotive division, Capgemini Tunisia

Works on vehicle software systems and on embedded engineering.

Battery Management Systems team

Develops and validates the embedded software of battery management functions for electric and hybrid vehicles.

My job: BMS QA tester for BMW Group

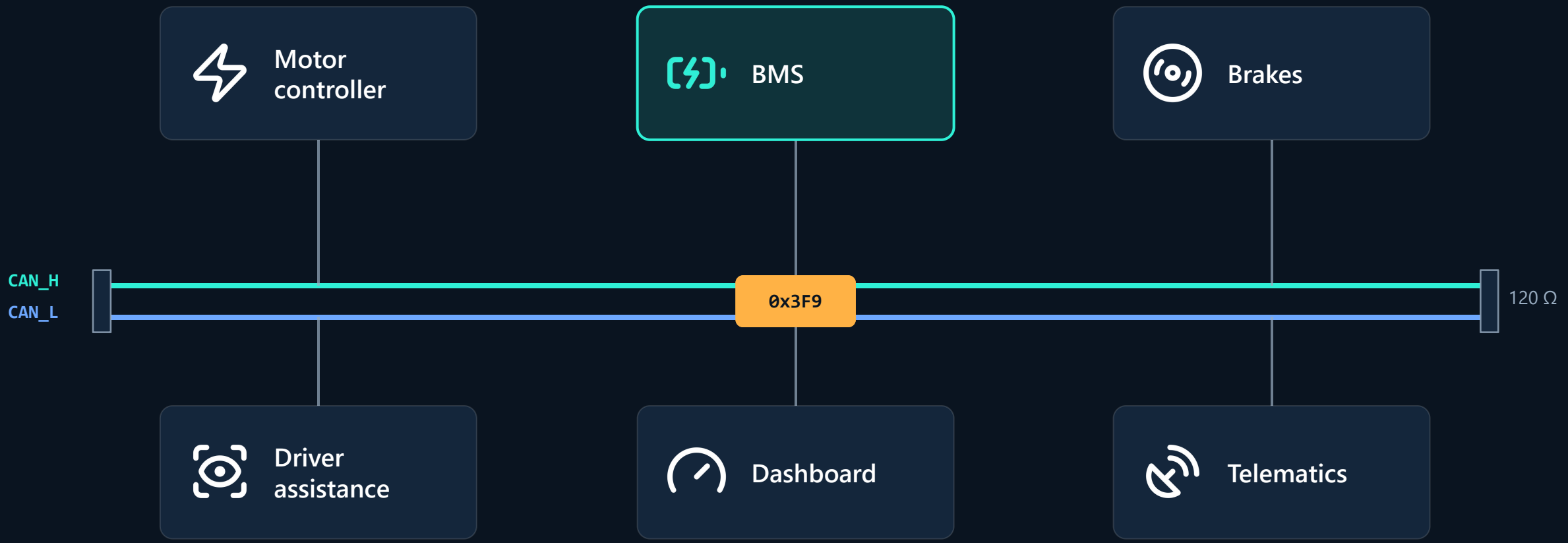
Work and study program since September 2024: full time, five days a week, with classes at ESPRIT on Saturdays.

0x100 · CONTEXT

How do the computers of a car talk to each other?

01 02 03 04 05 06 07 08

How do the computers of a car talk to each other?



Each ECU broadcasts short frames on one shared two-wire bus, and every node receives every frame.

How do the computers of a car talk to each other?

Identifier
11 bits: what the frame is, and its priority

Length
number of data bytes

Data
up to 8 bytes; their meaning depends on the vehicle (DBC file)

CRC
error check

BMS → `0x3F9  4E 00 00 00 00 00 00 00`

Unknown device → `0x3F9  4E 00 00 00 00 00 00 00`

=

Same identifier, same bytes.
Nothing in the frame tells the receivers who sent it.

NO SENDER IDENTITY
A frame does not say which ECU sent it.

NO AUTHENTICATION
Any node can send any identifier.

NO ENCRYPTION
Every node reads every frame.

The bus cannot tell a genuine frame from a forged one, or from a faulty one.

CAN protocol: ISO 11898. Shown on real cars by Koscher et al. (2010) and Miller and Valasek (2015).

0x200 · PROBLEM

What are the problems?

01 02 03 04 05 06 07 08

0x200 · PROBLEM

What are the problems?

OPEN ISSUES · day to day BMS validation work

5 OPEN

TIME	TYPE	PROBLEM	WHAT HAPPENS TODAY
00:01	WARNING	NO LIVE VIEW OF THE TRAFFIC	Raw frames say what happened, not whether it should have happened.
00:02	WARNING	WEAK REPLAY	Tools replay frames without judging them, or accept only a few log formats.
00:03	WARNING	SCATTERED TOOLS	An analyser, a diagnostic scanner and a spreadsheet, for one investigation.
00:04	WARNING	ALERTS WITHOUT EXPLANATION	A bare number, with no cause and no next step.
00:05	SECURITY	CAN HAS NO SECURITY	A toolchain that does not watch the bus cannot notice a malicious frame any more than a faulty one. A real but secondary motivation.

0x200 · PROBLEM

What already exists?

01 02 03 04 05 06 07 08

What already exists?

CAN analysers

GOOD AT

Capture, DBC decoding and replay. The professional standard (CANoe, CANalyzer).

MISSING

No judgement on whether a frame is normal for this vehicle.

Intrusion detection research

GOOD AT

Learns normal traffic and flags what deviates from it.

MISSING

One signal, aims at attacks, rarely transfers to another vehicle, no explanation.

Open source viewers

GOOD AT

Lightweight and open source: SavvyCAN, candump.

MISSING

Little control over replay, no notion of abnormal.

Diagnostic scanners

GOOD AT

Standard requests (OBD, UDS) to known ECUs.

MISSING

Only see what they ask for, not the whole bus.

Each tool covers one part. None of them combines decoding, replay, live monitoring, detection and explanation in one workflow.

PROBLEM STATEMENT

How can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts?

… and prove that it works on real recordings.

PROBLEM STATEMENT

How can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts?

… and prove that it works on real recordings.

ONE VIEW OF THE TRAFFIC

ANSWERS

- NO LIVE VIEW
- WEAK REPLAY
- SCATTERED TOOLS

ADAPTED TO EACH VEHICLE

ANSWERS

- MODELS RARELY TRANSFER

from one vehicle to another

EXPLAINS ITS OWN ALERTS

ANSWERS

- ALERTS WITHOUT EXPLANATION

a bare number, with no cause

0x200 · PROBLEM

How can we give a BMS engineer one view of the CAN traffic, adapted to each vehicle, that explains its own alerts?



THE IDEA

CANvisionNative

One platform where a BMS engineer replays, monitors and understands the CAN traffic of a vehicle.

One view
replay, live monitoring and detection together

Adapted to each vehicle
its own models and its own DBC file

Explains its own alerts
in plain language, next to the score

0x300 · OBJECTIVES AND REQUIREMENTS

What are the objectives?

01 02 03 04 05 06 07 08

What are the objectives?

GENERAL OBJECTIVE

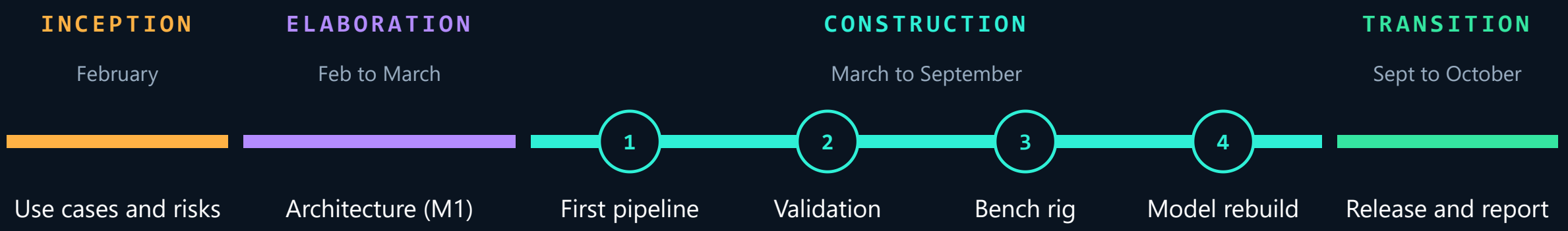
Design, build and validate a diagnostic platform for Battery Management Systems that gives an engineer one view of the CAN traffic, adapts to the vehicle being analysed, and explains its own alerts.

SPECIFIC OBJECTIVES

1. A detection pipeline that combines a machine learning model with timing, payload and persistence checks
2. The common log formats (CSV, MF4, candump, Vector, PCAN), a live simulator and live capture
3. One model per vehicle, and in production only the models evaluated on a recording kept out of training
4. A desktop client to import a log, replay it, review the anomalies and export a report
5. An explanation layer, separate from detection, with rules and a language model
6. A validation on real recordings with reproducible evidence, and every defect found corrected

How did I organize the work?

The Unified Process



Why: iterative, driven by risks, architecture first.

Two unknowns at the start: how well a model generalises, and how many defects validation finds.

0x300 · OBJECTIVES AND REQUIREMENTS

What must the platform do?

01 02 03 04 05 06 07 08

What must the platform do?

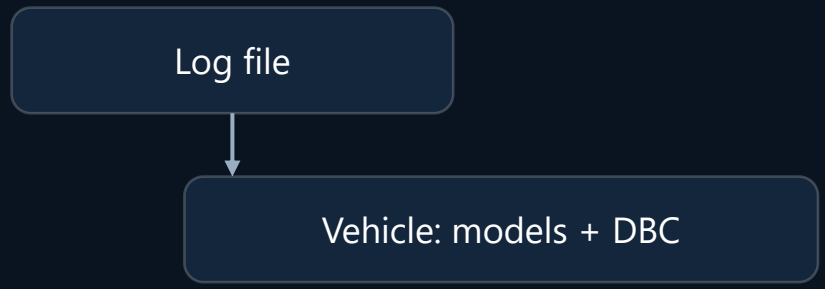
Import a recorded log

CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

Choose the vehicle

Its own models score the frames; its DBC file decodes them

HOW THEY CHAIN



What must the platform do?

Import a recorded log

CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

Choose the vehicle

Its own models score the frames; its DBC file decodes them

Replay the log

From 0.5× to 32× real time, with progress

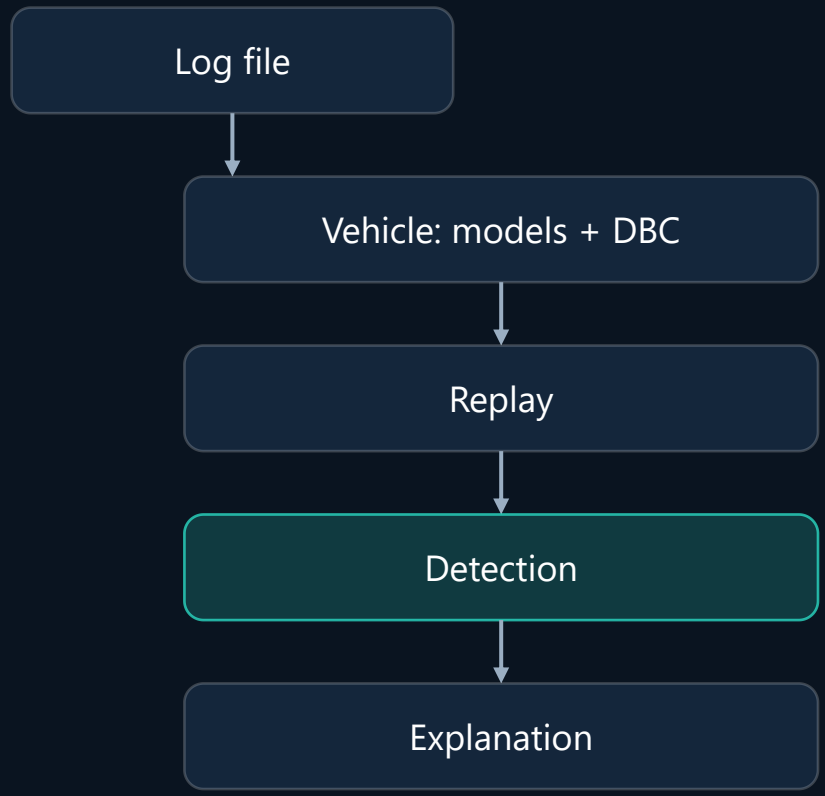
Detect anomalies

Score each frame; report what cannot be scored, with the reason

Explain an alert

Dominant signal, likely cause, what to check

HOW THEY CHAIN



What must the platform do?

Import a recorded log
CSV, MF4, Vector .asc, candump .log, PCAN .txt into one frame stream

Choose the vehicle
Its own models score the frames; its DBC file decodes them

Replay the log
From 0.5× to 32× real time, with progress

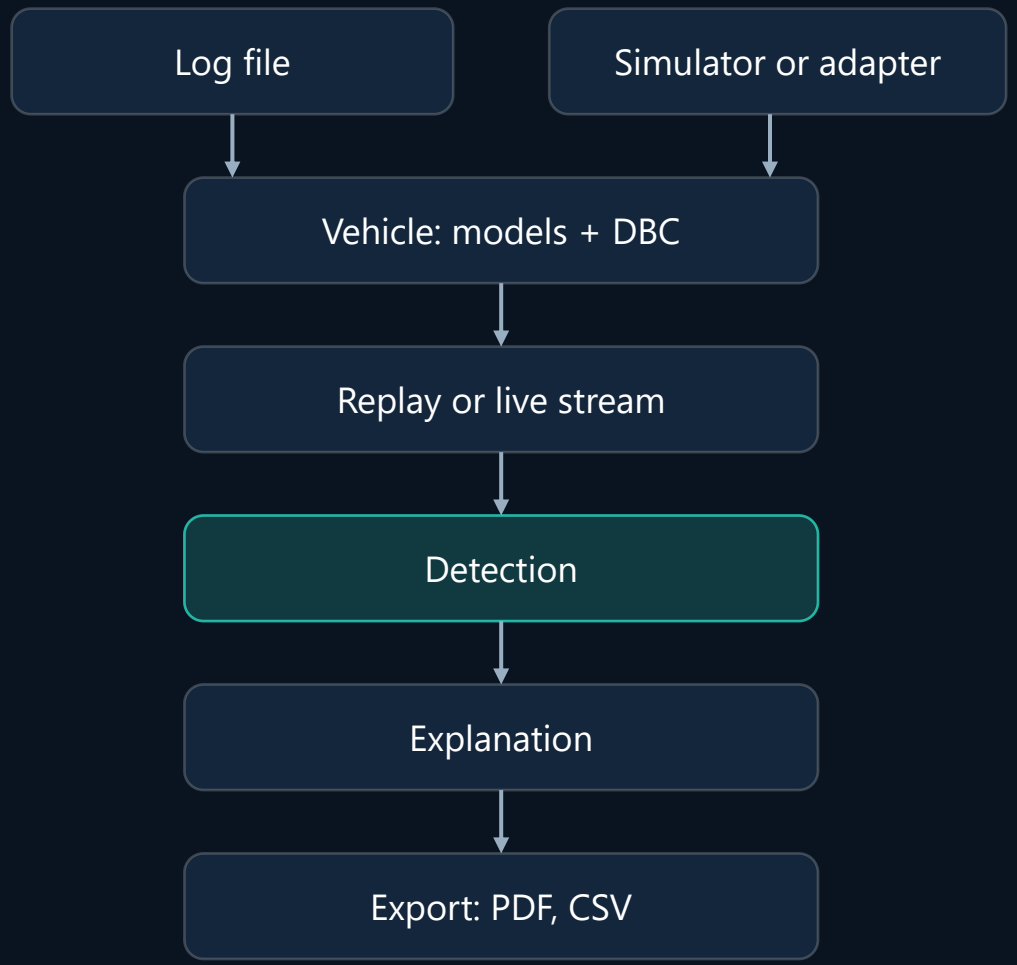
Detect anomalies
Score each frame; report what cannot be scored, with the reason

Explain an alert
Dominant signal, likely cause, what to check

Go live
Simulator, or a CAN adapter through python-can; record a session

Export the results
CSV file and PDF report

HOW THEY CHAIN

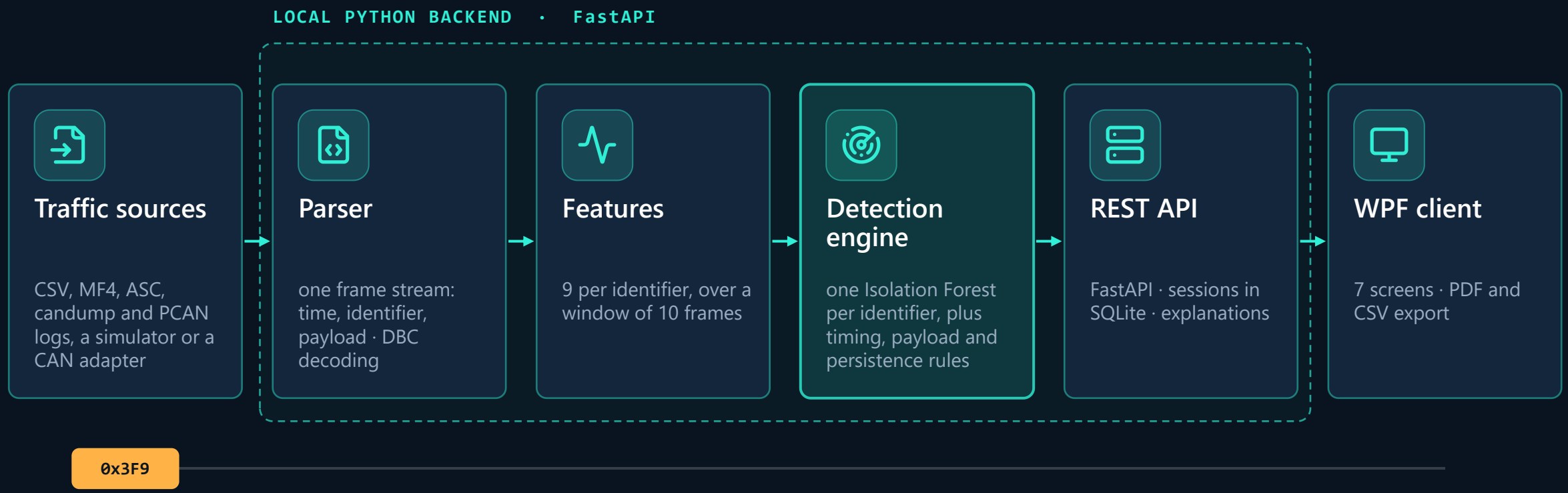


0x400 · SOLUTION

What is the proposed solution?

01 02 03 04 05 06 07 08

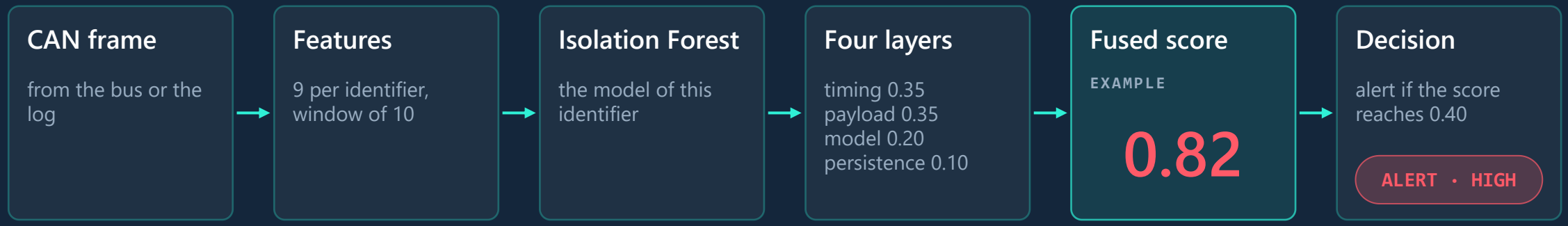
What is the proposed solution?



Two programs on one machine: a local Python backend and a Windows client, talking over HTTP.

What is the proposed solution?

How detection works: one frame, from the bus to a decision



Unscored frames are reported, never hidden. A frame with no model (warmup, sparse, unmodelled or unseen identifier) is counted as unresolved, with its reason.

Offline analysis of a whole recording: same features and models; each frame is flagged when the score of its model passes 0.5.

What is the proposed solution?

One model per CAN identifier

Frames of one identifier

here 0x3F9, in time order

```
35.000  4E 00 00 00
35.200  4E 00 00 00
35.400  4E 00 00 00
35.600  4E 00 00 00
35.800  4E 00 00 00
```

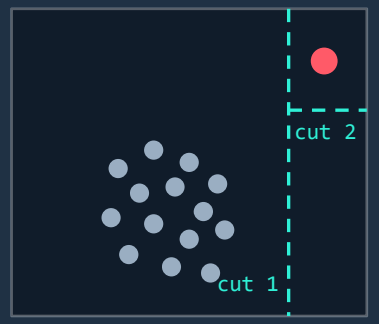
9 features

TIMING ×3
time between frames, its rolling mean and std

PAYLOAD ×6
byte change, its rolling mean, std and max; changed bytes; entropy

over a window of 10 frames

Isolation Forest



200 trees. An unusual frame is isolated in very few cuts.

A score from 0 to 1



The threshold, 0.5, is set so that 2 % of the training rows pass it. Above it, the frame is flagged.

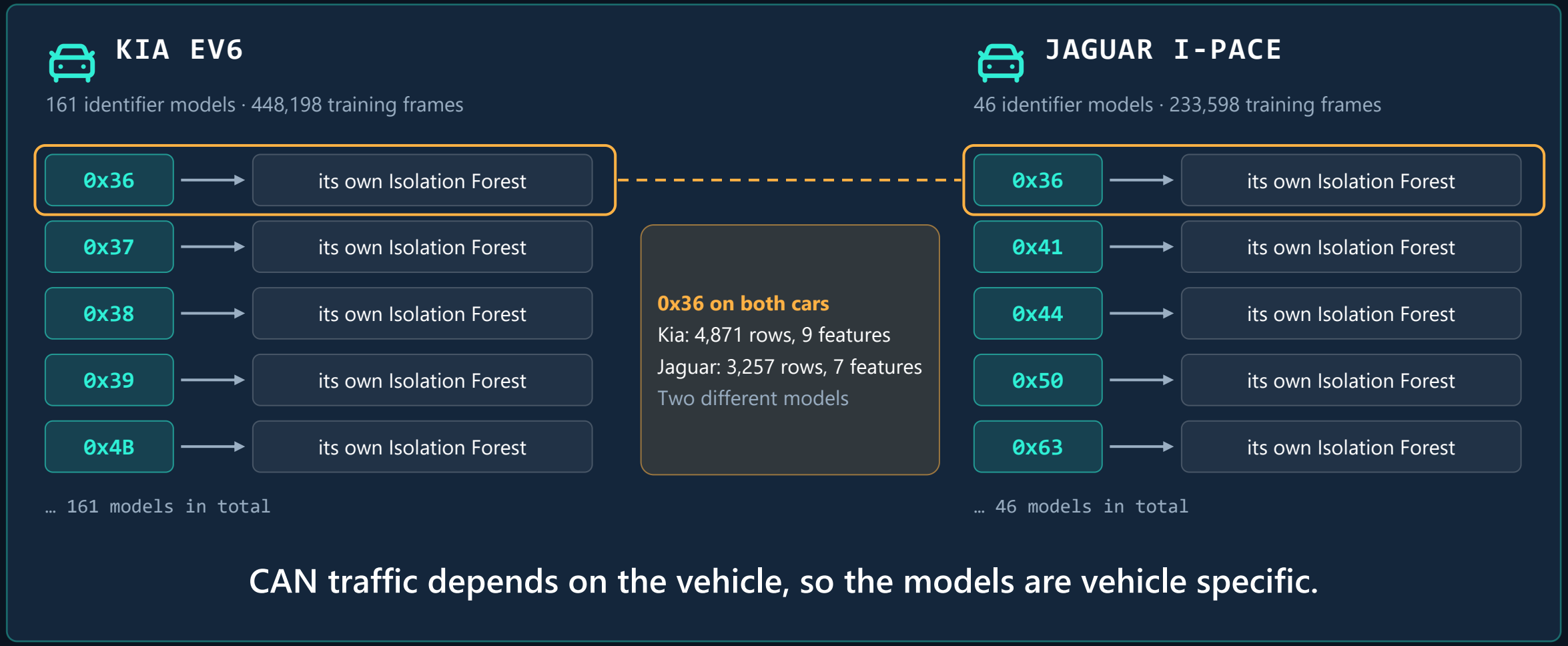
At least 1,000 training rows per identifier, or no model

Not scored, and said so: warmup, sparse, unmodelled or unseen identifier

The timing features carry 64 % of the splits

What is the proposed solution?

Vehicle-specific models: one model per CAN identifier, per vehicle



CAN traffic depends on the vehicle, so the models are vehicle specific.

What is the proposed solution?

Explanation: a supporting feature

ALERT

from the detection engine

0x063 · score 0.535

→

CONTEXT

decoded signals, layer scores, temporal pattern, knowledge base

→

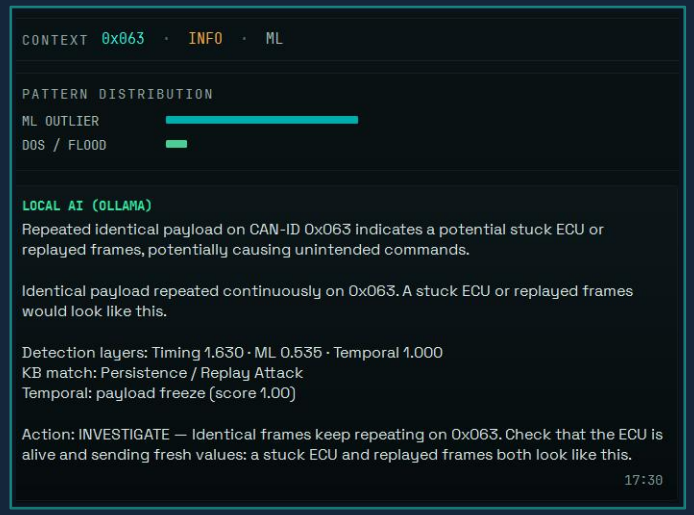
EXPLAINER

Gemini · Ollama · rules

→

DIAGNOSIS

dominant signal, likely cause, what to check



In the application: the explanation of the alert on 0x063, written by the local model

AI explains.

The detection engine decides.

DETECTION ENGINE scores, flags and severities

EXPLANATION words only: it never changes a score, a flag or a severity, and its numbers are checked against the alert

0x500 · DEMO

Can we see it working?

01 02 03 04 **05** 06 07 08

DEMONSTRATION

REAL APPLICATION.

0x600 · VALIDATION

How do we know it works?

01 02 03 04 05 **06** 07 08

How do we know it works?

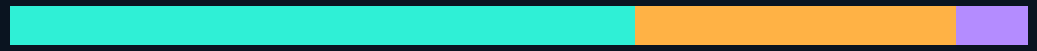
Iteration 2: the first pipeline on a one hour Kia EV6 recording (97,741 frames)

15,945

alerts, 16.3 % of the frames



Instead of raising the threshold, we attributed every alert to the layer that dominated it:



- **Machine learning 61.42 %** wrong model, wrong driving state
- **Timing 31.51 %** irregular UDS polling
- **Payload 7.07 %** real sensor variation

Six silent defects

1. The selected vehicle model was never used for scoring
2. Driving state: 99.93 % "parking" for an hour of driving
3. Diagnostic fields dominated the cause shown to the analyst
4. Replay formats other than CSV bypassed the common parser
5. A polling timer stayed stopped after an import
6. The explanation cache returned another session's text

None of them crashed. Each appeared only when the output was compared with a fact known independently, such as the car being driven.

How do we know it works?

Iteration 2: six corrections, the same recording, the same threshold

11,971

alerts after the corrections



Saturated model score (≥ 0.999)	77.6 to 78.6 % → **0.0 %**
Replay throughput	≈ 143 frames/s → **≈ 194.6 frames/s**
Driving state	99.93 % parking → **99.92 % driving**

Six defects, six corrections

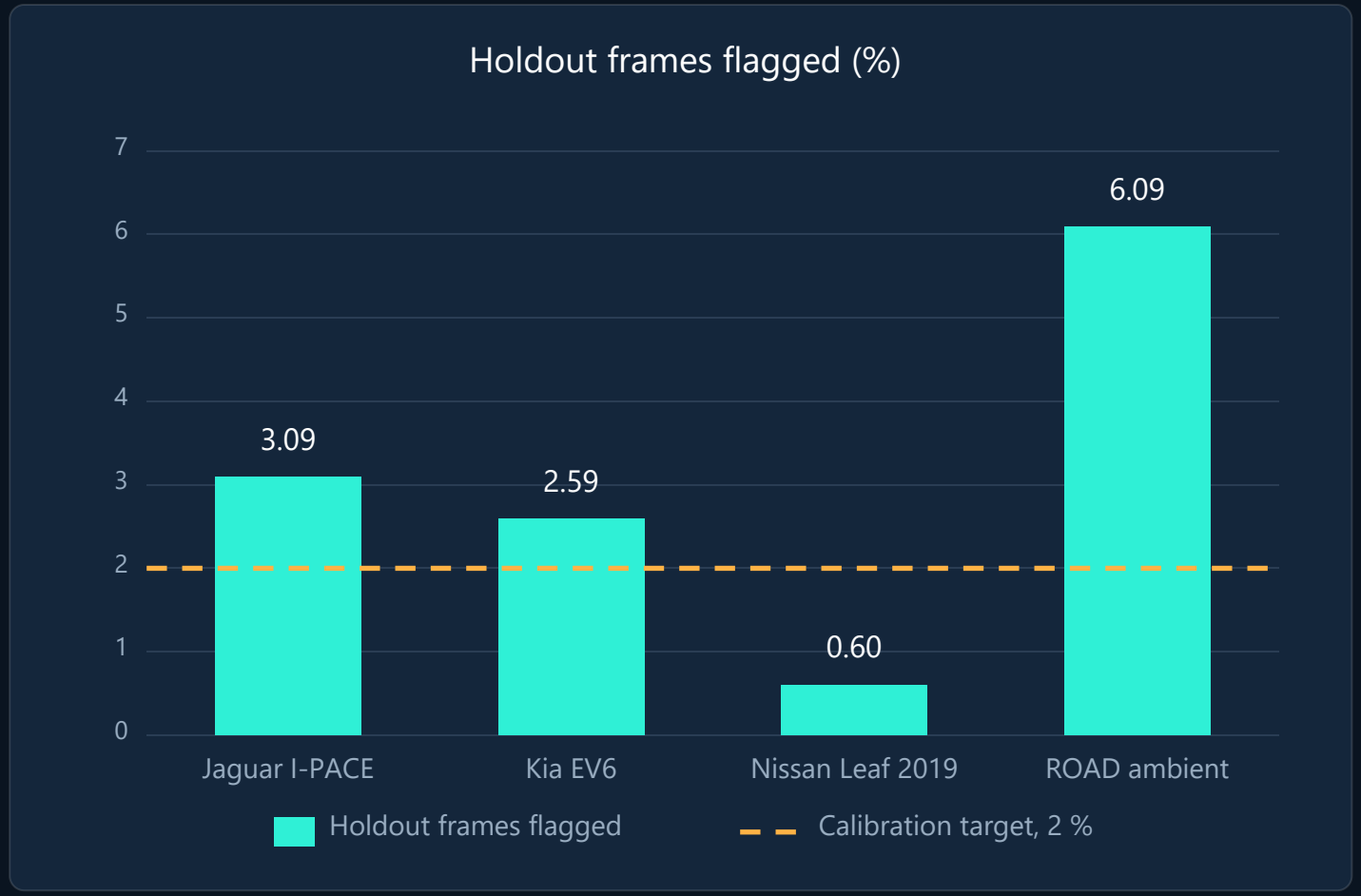
- ✓ The selected vehicle model was never used for scoring
- ✓ Driving state: 99.93 % "parking" for an hour of driving
- ✓ Diagnostic fields dominated the cause shown to the analyst
- ✓ Replay formats other than CSV bypassed the common parser
- ✓ A polling timer stayed stopped after an import
- ✓ The explanation cache returned another session's text

The threshold was never touched. The design stayed fragile, so I rebuilt every model with a stricter pipeline in iteration 4.

Counts measured after the four data-driven fixes (1 to 4); fixes 5 and 6 were in the client and the explanation cache.

How do we know it works?

Iteration 4: the final models on recordings they never saw



296 identifier models
for 4 vehicles

3,436,061 training frames
1,130,500 holdout frames, no overlap

ROAD at 6.09 %: ID 0x193 never changes in training but changes on the highway. A coverage gap, not a threshold problem.

Held back: MG ZS EV, 43.6 % flagged, with 78 % of its holdout on identifiers it never saw.

No attack labels: a flagged rate on unlabelled recordings is not a false positive rate, a precision or a recall.

0x700 · LIMITS

What are its limits?

01 02 03 04 05 06 **07** 08

What are its limits?

LIVE CAPTURE →

Tested on a bench rig only, not on a vehicle.
The transport of live frames is validated; detection on a live vehicle bus is not measured.

LIVE PERFORMANCE →

About 200 to 260 frames per second.
Enough for a slow bus such as the Nissan Leaf (about 100 frames/s), not for buses that carry thousands.

HOLDOUT DATA →

No attack labels.
We report flagged rates, not a detection rate, a precision or a recall.

VEHICLE COVERAGE →

Rare identifiers get no model.
Jaguar I-PACE: 82.6 % of the holdout frames scored. One model held back (MG ZS EV).

Each limit is measured and stated in the report.

0x800 · CONCLUSION

What can we conclude?

01 02 03 04 05 06 07 **08**

What can we conclude?

DELIVERED

- A Windows client and a Python backend: recorded logs, a live simulator and live capture
- 4 vehicle models, 296 identifier models, evaluated on 1,130,500 frames they never saw
- A training pipeline that refuses a model failing its evaluation
- Explanations in plain language, kept apart from detection
- A bench rig with its firmware, and 318 + 86 automated tests

WHAT I LEARNED

- The most costly defects were the silent ones: none of them crashed.
- A holdout is only meaningful when the split is made by recording, not by frame.
- A flagged rate on unlabelled data is not a detection rate.

"A detection system is only as trustworthy as the evidence behind each of its numbers."

What are the perspectives?

01

Live capture on a real vehicle

The first step: it closes milestone M2.

02

Labelled attacks

Evaluate the final models on recordings with labelled attacks, such as the ROAD dataset, to measure what they detect as well as what they flag.

03

A faster live path

Score several frames at once or use fewer trees, with each change measured on the holdout recordings.

04

More recordings per vehicle

A holdout for every vehicle, and a model for more of the rare identifiers.

CANvisionNative

DIAGNOSE . MONITOR . DETECT . EXPLAIN . VALIDATE

From CAN frames to explainable diagnostics.

Thank you for your attention. **Questions?**

esprit Se former autrement | Capgemini engineering

How should it behave?

Performance

Offline: hundreds of thousands of frames in a few minutes. Live: keep up with the bus.

Reliability

A defect is fixed with evidence, never by tuning a threshold until the numbers look better.

Usability

No machine learning background needed to read an alert and its explanation.

Format independence

Every log format enters the pipeline as the same frame stream.

Extensibility

A new vehicle needs only a DBC file and a model folder that passes the promotion checks.

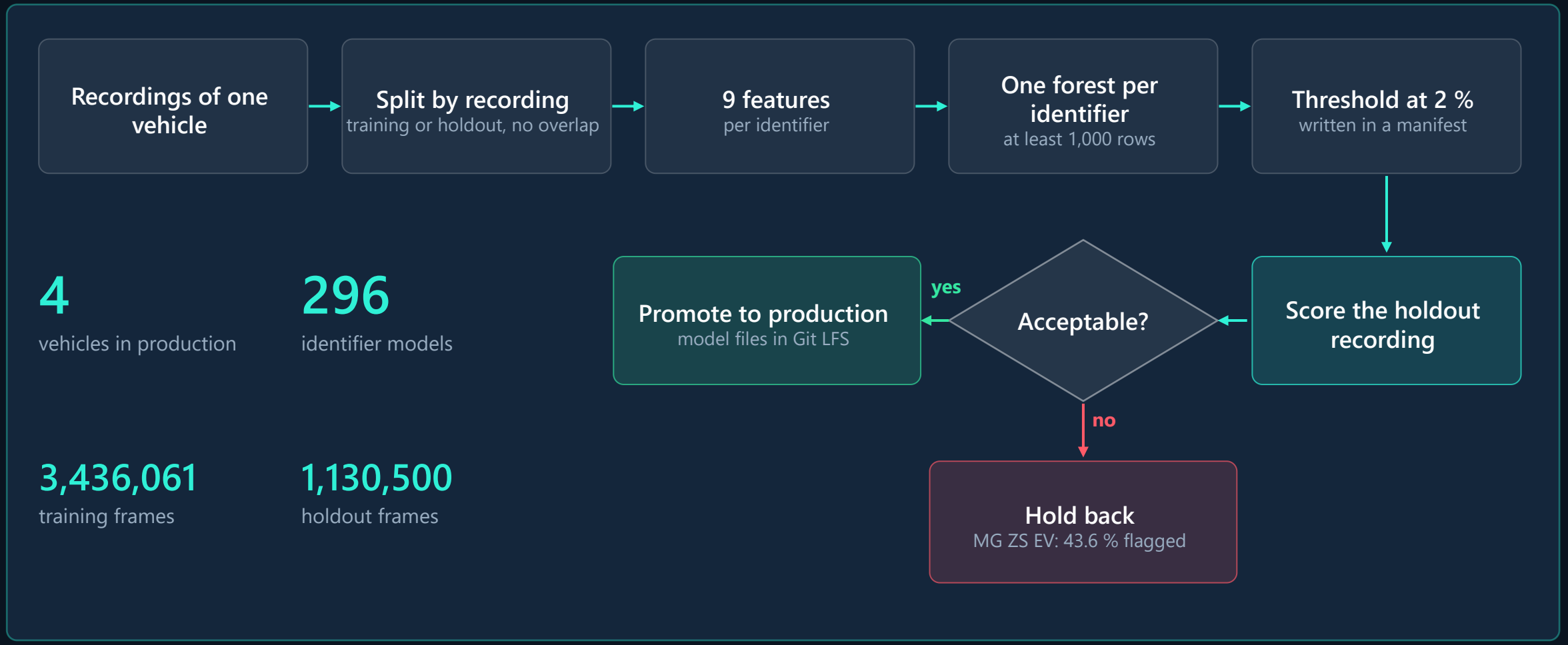
Graceful degradation

No language model reachable: a rule based explanation instead of a failure.

Platform: Windows 10 or 11 with .NET Framework 4.8 · a local Python 3.11 backend · a python-can adapter, only for live capture

What is the proposed solution?

From recordings to a model in production



4
vehicles in production

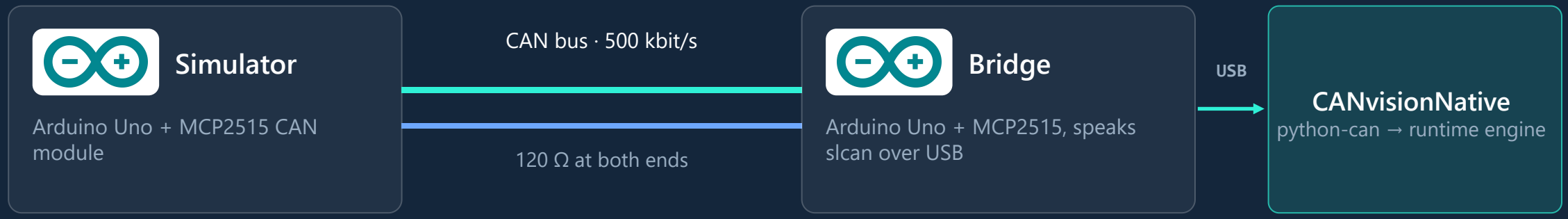
296
identifier models

3,436,061
training frames

1,130,500
holdout frames

What is the proposed solution?

Live capture without a vehicle: the bench rig



WHAT THE SIMULATOR SENDS

0x316	speed, 60 km/h	every 20 ms
0x3F9	state of charge, 78 %	every 200 ms
0x666	should not exist	every 20 s, from 35 s

About 55 frames per second

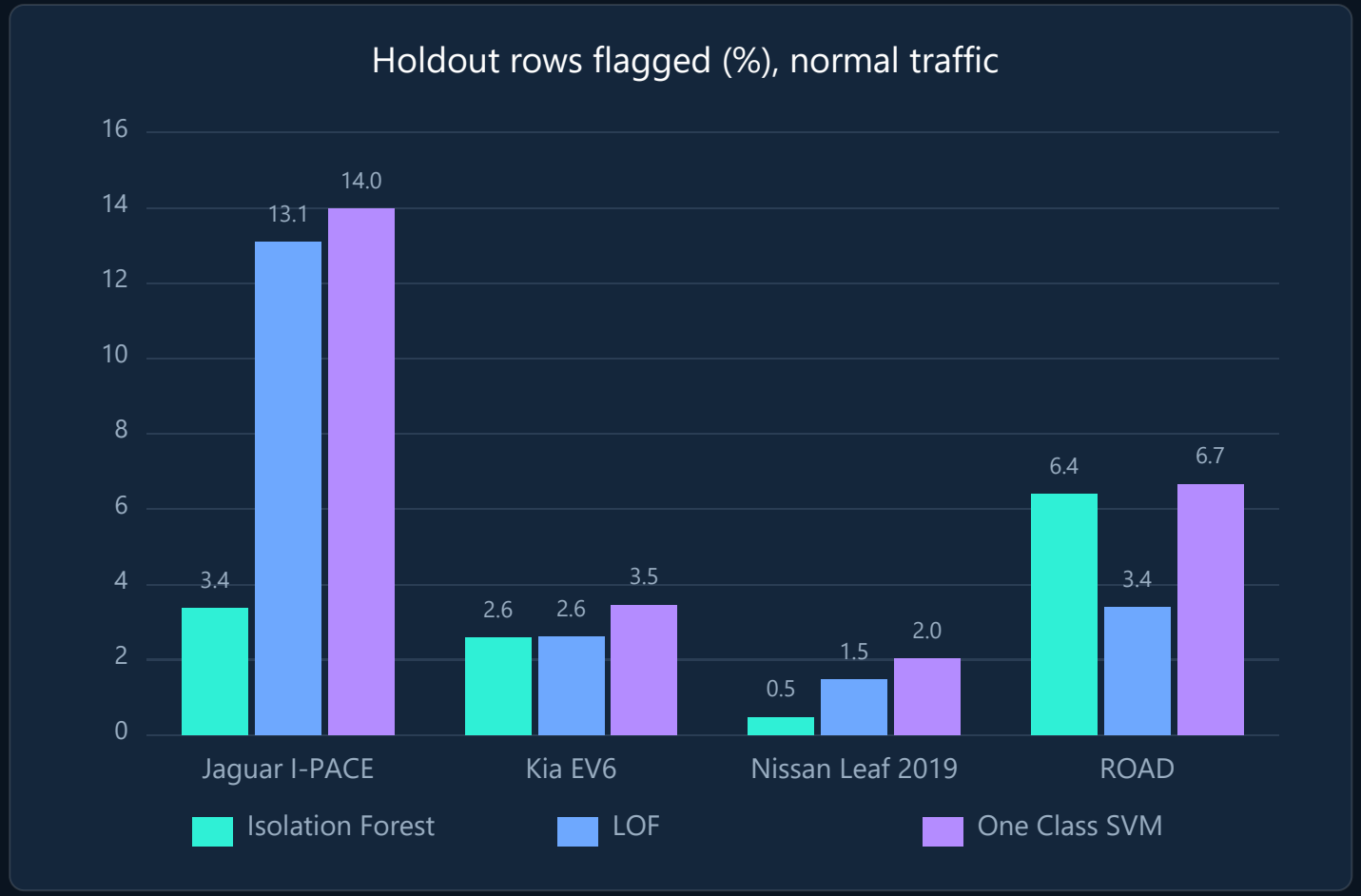
What it shows, and what it does not

It tests the transport of live capture, from the bus to the runtime engine.

It does not measure detection: the traffic is synthetic and much simpler than a car's. No vehicle was available.

How do we know it works?

Why Isolation Forest? Compared with LOF and a One Class SVM



MEAN SHARE FLAGGED

Isolation Forest	LOF	One Class SVM
3.2 %	5.1 %	6.5 %

Lowest worst case: 6.4 % for Isolation Forest, against 13.1 % and 14.0 %.

The price: it is the slowest. 23.4 ms to score a frame against 0.97 and 0.42 ms; 138 s of training against 41 and 17 s.

We kept it: the offline analysis, the main use of the platform, needs few and even false alarms more than speed.

Same split, same 9 features, same training rows (at most 5,000 per identifier), same threshold rule. No labels.

0x600 · VALIDATION

How do we know it works?

Speed, live capture and automated tests

Speed

≈ 24 ms

per model call

Live path: 200 to 260 frames/s

Keeps up with the Nissan Leaf (about 100 frames/s), not with the Jaguar I-PACE (about 3,000)

Offline: 491,969 frames in 187 s

Live capture on the rig

16,304

frames, then one malformed serial line closed the session

Fixed: the bad line is skipped and logged; the link then held at about 55 frames/s

Fixed: an unplugged rig kept showing old data; the link is now lost after 5 s without frames

Automated tests

318 + 86

backend tests (pytest) and desktop tests (xUnit)

Run after every change: parsers, manifests, model status, scoring, unresolved frames, calibration, PDF report